PROJECT REPORT

E-WASTE TO BEST

Smart E-Waste Recycling & Metal Recovery System

Submitted in partial fulfilment of the requirements for the academic project

Mahatma Gandhi Mission's College of Engineering and Technology

Submitted by
Pawan Poojari

Project Guide
Sudharsan Bhasale

ACADEMIC YEAR 2026–2027

Department: _____
Institute / University submission copy

CERTIFICATE

This is to certify that the project report entitled “Smart E-Waste Recycling & Metal Recovery System”, developed as “E-WASTE TO BEST”, is submitted by Pawan Poojari of Mahatma Gandhi Mission's College of Engineering and Technology for the academic year 2026–2027, under the guidance of Sudharsan Bhasale.

The report describes the project scope, system analysis, architecture, database design, implementation modules and a proposed testing plan based on the supplied source package. Final certification is subject to institutional review and confirmation of project execution and testing.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

DECLARATION

I, Pawan Poojari, declare that this project report entitled “Smart E-Waste Recycling & Metal Recovery System” has been prepared for academic submission based on the project source files and documentation supplied for E-Waste to Best. The report is intended to describe the system faithfully. Theoretical recovery values are identified as model estimates, and test cases are not represented as passed unless execution evidence is available.

Any institutional declarations, originality statements or prescribed formatting requirements should be reviewed and signed according to the college’s official submission rules.

Signature of Student: _____

Date: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, Sudharsan Bhasale, for guidance and academic support throughout the preparation of this project. I am thankful to Mahatma Gandhi Mission's College of Engineering and Technology for providing the academic environment in which this work could be developed.

I also acknowledge the developers and maintainers of PHP, MySQL/MariaDB, Bootstrap, Chart.js and XAMPP, whose technologies support the application. Finally, I thank everyone who contributed feedback and encouragement during the preparation of the project and its documentation.

ABSTRACT

E-Waste to Best is a web-based Smart E-Waste Recycling and Metal Recovery System designed to support responsible electronic-waste handling through information, estimation and collection coordination. The application provides public awareness pages, repair and reuse guidance, a theoretical metal-recovery and value estimator, a collection-centre directory, citizen registration and login, pickup scheduling, request history, environmental impact visualizations and administrator tools. The server-side application is implemented in PHP 8.x using PDO for database access, while MySQL/MariaDB stores user accounts, collection centres, device-category coefficients, metal rates, pickup requests and impact logs. HTML5, CSS3, JavaScript, Bootstrap and Chart.js support the user interface and interactive visualizations. A key design principle is transparency between theoretical projections calculated from configurable coefficients and actual values recorded after physical collection and verification. The report describes the system architecture, functional requirements, database relationships, module implementation and test plan. The supplied archive was inspected as source code; a live runtime test was not performed during report generation, so test cases are presented as acceptance criteria rather than claimed results. The platform is intended as an academic prototype and requires validation of scientific coefficients, collection-centre records, security controls and deployment practices before real-world operation.

Keywords: E-waste, recycling, metal recovery, PHP, MySQL, pickup scheduling, environmental impact, verification.

TABLE OF CONTENTS

Chapter No.	Chapter Title	Sub-topics / Sections
—	**Preliminary Pages**	Certificate
		Declaration
		Acknowledgement
		Abstract
Chapter 1	**Introduction**	1.1 Background
		1.2 Project Overview
		1.3 Problem Statement
		1.4 Aim
		1.5 Objectives
		1.6 Scope
		1.7 Intended Users
		1.8 Expected Benefits
		1.9 Report Organization
Chapter 2	**Survey of Technologies**	2.1 Web Application Model
		2.2 HTML5 and CSS3
		2.3 JavaScript
		2.4 Bootstrap
		2.5 PHP
		2.6 PDO
		2.7 MySQL
		2.8 Chart.js
		2.9 XAMPP
		2.10 Technology Comparison
		2.11 Related Work
Chapter 3	**Requirements and System Analysis**	3.1 Existing Process
		3.2 Proposed System
		3.3 Functional Requirements
		3.4 Non-Functional Requirements
		3.5 Roles and Permissions
		3.6 Use Cases
		3.7 Methodology
		3.8 Requirements
		3.9 Feasibility
		3.10 Assumptions
Chapter 4	**System Design**	4.1 Architecture
		4.2 Logical Layers
		4.3 Request Flow

TABLE OF CONTENTS

Chapter No.	Chapter Title	Sub-topics / Sections
Chapter 4 (cont.)	System Design (cont.)	4.4 Database Design
		4.5 ER Summary
		4.6 Data Dictionary
		4.7 Estimation Logic
		4.8 Formulae
		4.9 Pickup Lifecycle
		4.10 Interface
		4.11 Security
		4.12 Limitations
Chapter 5	Implementation and Module Description	5.1 Structure
		5.2 Home
		5.3 Awareness and Reuse
		5.4 Estimator
		5.5 Centre Locator
		5.6 Authentication
		5.7 Citizen Dashboard
		5.8 Pickup and History
		5.9 Impact
		5.10 Admin Dashboard
		5.11 Verification
		5.12 Administration
		5.13 Algorithms
		5.14 Request Codes
		5.15 Installation
		5.16 Notes
Chapter 6	Testing and Results Discussion	6.1 Testing Approach
		6.2 Test Cases
		6.3 Calculation Validation
		6.4 Security Validation
		6.5 Source Inspection Findings
		6.6 Risks to Verify
		6.7 Discussion
Chapter 7	Conclusion and Future Scope	7.1 Conclusion
		7.2 Achievements
		7.3 Limitations
		7.4 Future Enhancements
		7.5 Final Remarks
—	References and Appendices	References
		Appendix A: Installation Checklist
		Appendix B: Project File Map
		Appendix C: Project Screenshots and Explanation
		Appendix D: Selected Source Code and Explanation
		Appendix E: Formulas, Terms and Interpretation

CHAPTER 1: INTRODUCTION

1.1 Background

Electronic and electrical equipment is replaced at an increasingly fast pace. End-of-life phones, computers, printed circuit boards, cables, displays and storage devices contain both reusable components and materials that require controlled handling. When discarded with ordinary household waste, electronic waste (e-waste) can create environmental and health risks. At the same time, electronic assemblies may contain recoverable metals such as copper, aluminium, silver and gold. Responsible collection, repair, reuse and recycling can therefore support resource conservation and reduce inappropriate disposal.

1.2 Project Overview

E-Waste to Best is a web-based platform designed to connect citizens with collection centres and make the potential resource value of discarded electronics easier to understand. The application combines public awareness content, reuse guidance, a theoretical metal recovery estimator, a collection-centre directory, citizen registration and login, pickup scheduling, request tracking, administrative workflow management, metal-rate configuration, and post-collection verification records.

1.3 Problem Statement

People may not know where to take unwanted electronics, whether an item should be repaired or recycled, or what materials could potentially be recovered. Collection processes may also lack a simple digital mechanism for booking, status tracking and documenting the difference between an initial estimate and measurements recorded after physical inspection. E-Waste to Best addresses these problems through one web interface and a database-backed pickup workflow.

1.4 Aim

To develop a responsive web application that supports e-waste awareness, theoretical metal recovery estimation, collection-centre discovery, pickup booking, administrative coordination and recording of verified recovery data.

1.5 Objectives

- Provide accessible information about e-waste hazards and responsible disposal.
- Offer reuse and refurbishment suggestions before an item is sent for recycling.
- Estimate potential quantities and benchmark values for gold, silver, copper and aluminium using configurable device-category coefficients.
- Allow registered citizens to create and monitor pickup requests.
- Enable administrators to assign collection centres, update request status and record post-collection measurements.
- Maintain structured records for users, collection centres, device categories, metal rates, pickup requests and environmental impact logs.
- Clearly distinguish theoretical estimates from physically verified results.

1.6 Scope

The project is scoped as a local PHP/MySQL web application intended to run in a XAMPP environment. It includes public pages, citizen and administrator areas, and a relational database. The centre directory can store address, contact details, operating hours, accepted materials and coordinates. The application can display indicative calculations and store verified values entered by an administrator after collection. The current project package does not demonstrate integration with laboratory instruments, payment gateways, courier tracking hardware or a live commodity-price API; such capabilities should not be assumed.

1.7 Intended Users

- Public visitors seeking e-waste guidance, reuse suggestions or a recovery estimate.
- Registered citizens who create pickup requests and review their request history.
- Administrators who manage users, collection centres, benchmark metal rates and pickup workflows.
- Collection-centre or laboratory personnel whose measurements may be entered by an authorized administrator.

1.8 Expected Benefits

The platform is intended to make responsible disposal more understandable, reduce friction in requesting collection, improve visibility of request status, and keep estimates separate from actual intake measurements. Its educational content also encourages repair and reuse where practical. Environmental totals are indicative model outputs and should not be interpreted as independently audited impact measurements unless validated data and calculation methods are documented.

1.9 Organization of the Report

Chapter 1 introduces the problem and objectives. Chapter 2 surveys the technologies used. Chapter 3 presents requirements and analysis. Chapter 4 explains architecture, database design and workflows. Chapter 5 describes implementation modules and representative algorithms. Chapter 6 defines testing and discusses the evidence available from the supplied source package. Chapter 7 concludes with limitations and future enhancements. References and appendices follow the chapters.

CHAPTER 2: SURVEY OF TECHNOLOGIES

2.1 Web Application Model

The application follows a client-server model. A user interacts through a web browser. The browser renders HTML and CSS and runs JavaScript for interactive previews. PHP processes server requests and database operations. MySQL stores persistent records. Apache, supplied through XAMPP, serves the PHP application in the intended local development environment.

2.2 HTML5 and CSS3

HTML provides the semantic structure of forms, tables, cards, navigation and content pages. CSS provides layout, responsive behaviour, typography and the eco-technology visual theme. The supplied stylesheet includes reusable visual patterns for the public interface and dashboards.

2.3 JavaScript (ES6+)

JavaScript supports client-side interactions such as estimator previews, page behaviours and chart configuration. Client-side calculations improve responsiveness but are not treated as authoritative: the server-side PHP calculation routine is used for persisted booking estimates. Server-side validation remains necessary because browser code can be bypassed.

2.4 Bootstrap 5.3 and Bootstrap Icons

Bootstrap supplies responsive grid utilities, common form controls, navigation and component styling. Bootstrap Icons provide interface symbols. A responsive framework reduces repetitive layout work, although application-specific testing is still needed for small mobile screens, long tables and keyboard navigation.

2.5 PHP 8.x

PHP is used for request handling, session-based authentication, form validation, database access and business rules. The source uses a modular, MVC-lite organization rather than a full framework: configuration, shared includes, public pages, citizen pages and admin pages are separated into folders. Reusable helper functions support sanitization, flash messages, formatting and estimation.

2.6 PDO and Prepared Statements

PHP Data Objects (PDO) provides a database abstraction interface. The database connection configures exception-based error handling, associative fetches and native prepared statements. Parameterized queries are used in key data operations to reduce SQL injection risk. Prepared statements do not replace validation, access control or output encoding.

2.7 MySQL / MariaDB and InnoDB

The database is named ewaste_db and uses utf8mb4 character encoding. InnoDB supports relational constraints and transactions. Foreign keys connect pickup requests to users, collection

centres and device categories; impact logs reference pickup records. These relationships help preserve referential integrity.

2.8 Chart.js

Chart.js is used in the citizen impact view and administrator dashboard to visualize environmental and operational indicators. Charts depend on the data returned by the application; their labels should identify whether a value is estimated or verified, and they should not imply scientific certainty beyond the underlying inputs.

2.9 XAMPP

XAMPP packages Apache, PHP and MySQL/MariaDB for local development and demonstration. The README describes a Windows setup workflow using a batch launcher or manual placement under htdocs, starting Apache and MySQL, importing config/seed_data.sql through phpMyAdmin, and opening the local application in a browser.

2.10 Technology Comparison

Need	Selected technology	Reason in this project	Consideration
Page structure	HTML5	Widely supported browser markup	Semantic structure and accessibility still require care
Responsive UI	CSS3 + Bootstrap 5.3	Reusable grid and components	Custom CSS must be tested across viewports
Interactive previews	JavaScript ES6+	Immediate feedback without a full page refresh	Client values must be validated again on the server
Server logic	PHP 8.x	Direct fit with XAMPP and simple deployment	A larger system may benefit from a formal framework
Persistence	MySQL / InnoDB	Relational data and foreign keys	Backups and least-privilege database accounts are important
Database access	PDO	Parameterized queries and consistent access layer	Errors should not expose internal details in production
Charts	Chart.js	Simple browser-side visualizations	Chart data quality and labels determine interpretation

2.11 Related Work and Design Rationale

E-waste platforms commonly combine awareness, collection logistics and circular-economy messaging. The distinguishing design choice in this project is to place a transparent, theoretical recovery estimate alongside a pickup lifecycle and separate fields for post-collection verified quantities. This design is useful for academic demonstration because it shows that model estimates and physical measurements are different data classes. The project README references USGS and EPA materials as background for baseline coefficients; each coefficient should be traced to a precise source before the estimator is used for commercial or scientific decisions.

CHAPTER 3: REQUIREMENTS AND SYSTEM ANALYSIS

3.1 Existing Process and Limitations

A manual e-waste process may rely on general awareness, phone calls, informal drop-off information and paper-based records. Users may not receive a persistent request code or a clear status history. Estimates of recoverable materials can also be misunderstood as guaranteed payouts. These issues motivate a single information system with traceable records and explicit estimate-versus-verification fields.

3.2 Proposed System

The proposed system provides public information and a centre locator, then allows a registered citizen to select a device category, quantity, preferred date and time slot, and pickup address. The system calculates theoretical metal yields and an indicative value from stored coefficients and metal rates. An administrator can assign a centre and update the request status. After collection, actual weight, metal quantities, payout and verification notes can be recorded separately.

3.3 Functional Requirements

- FR-01: Display the home page and project navigation.
- FR-02: Display e-waste hazard and disposal-awareness information.
- FR-03: Display reuse and repair guidance for common electronic items.
- FR-04: Calculate theoretical metal quantities and indicative value for a selected category and quantity.
- FR-05: Search or filter collection centres using location-related fields and keywords.
- FR-06: Register users and authenticate users using stored password hashes.
- FR-07: Enforce separate citizen and administrator access paths.
- FR-08: Allow an authenticated citizen to create a pickup request with address, date, time slot, category and quantity.
- FR-09: Generate a human-readable request code and store calculated estimates with the request.
- FR-10: Display request history and request details to the citizen.
- FR-11: Allow administrators to filter requests, assign centres and update status/remarks.
- FR-12: Allow administrators to create, update, activate/deactivate and remove collection-centre records.
- FR-13: Allow administrators to update indicative metal rates.
- FR-14: Allow administrators to enter actual weight, metal recovery quantities and payout after physical verification.
- FR-15: Record environmental impact entries linked to completed pickup requests.
- FR-16: Present dashboard metrics and charts from database values or explicitly marked fallback/demo values.

3.4 Non-Functional Requirements

- Usability: navigation, labels, forms and status messages should be understandable to first-time users.

- Responsiveness: public and portal pages should remain usable on mobile, tablet and desktop viewports.
- Security: passwords must be hashed; protected pages must enforce role checks; POST forms should validate CSRF tokens; dynamic output should be encoded.
- Data integrity: foreign keys and server-side validation should prevent invalid relationships and nonsensical quantities.
- Maintainability: shared configuration and helper functions should avoid duplicated logic.
- Transparency: all recovery outputs should be labelled as theoretical until verification data is entered.
- Performance: common pages and filtered tables should respond efficiently for a college-scale dataset.
- Portability: the supplied installation path targets a local XAMPP environment.

3.5 User Roles and Permissions

Function	Visitor	Citizen	Administrator
Read awareness/reuse pages	Yes	Yes	Yes
Use public estimator / browse centres	Yes	Yes	Yes
Create pickup request	No	Yes	Administrative test accounts may access citizen functions only if routed accordingly
View own history/profile	No	Yes	No, unless using a citizen account
Manage all pickup requests	No	No	Yes
Manage centres and metal rates	No	No	Yes
Record physical verification	No	No	Yes
Manage user directory	No	No	Yes

3.6 Use Case Summary

Primary use cases are: browse public information; estimate material recovery; locate a collection centre; register and sign in; create a pickup request; review request history; update profile; assign and progress a pickup; manage centres; update benchmark rates; record verified recovery; and view impact dashboards. Each protected use case depends on the relevant session and role guard.

3.7 Development Methodology

The supplied blueprint groups implementation into five phases: database and configuration foundation; public pages and estimator; authentication and pickup booking; citizen/admin portals; and quality assurance/documentation. The blueprint's Gantt schedule assigns one-day tasks to these phases, but it should be treated as a planned sequence rather than proof of actual elapsed development time.

3.8 Hardware and Software Requirements

Category	Minimum / intended requirement
Operating system	Windows desktop or laptop for the supplied batch launcher; other systems may run the PHP stack with adapted setup
Web server	Apache 2.4 through XAMPP
Server-side language	

Database	MySQL or MariaDB with InnoDB support
Browser	Recent desktop or mobile browser with JavaScript enabled
Frontend libraries	Bootstrap 5.3, Bootstrap Icons and Chart.js (loaded by the page includes/CDN references)
Development hardware	A typical student laptop/desktop capable of running a local Apache/PHP/database stack
Network	Internet may be needed for external CDN assets and map links; local core processing is designed around XAMPP

3.9 Feasibility Analysis

- Technical feasibility: the stack is widely used for local PHP web development and is compatible with the supplied XAMPP workflow.
- Operational feasibility: citizens can use browser forms, while administrators use separate pages for operational tasks.
- Economic feasibility: the application uses common open-source technologies; operational costs may still arise from hosting, verified collection operations and laboratory assays.
- Schedule feasibility: modular pages allow staged implementation, though actual delivery time must be confirmed from project records.
- Environmental feasibility: the application supports awareness and collection coordination; real environmental benefit depends on responsible downstream handling and verified data.

3.10 Assumptions and Constraints

The system assumes a configured local database, valid category/rate records and an administrator who can record centre and verification data. Benchmark coefficients are stored as reference data and are not universally accurate for every device model or condition. The supplied package does not include evidence of a live lab integration or independent environmental audit. Sample/demo records are intended for demonstration and must be clearly distinguished from real operational records.

CHAPTER 4: SYSTEM DESIGN

4.1 Architectural Overview

E-Waste to Best uses a monolithic, modular PHP architecture with an MVC-lite separation. The browser presents the interface; shared includes provide navigation, reusable functions, authentication guards and CSRF utilities; page controllers implement module-specific request handling; PDO connects to MySQL; and database tables persist user, centre, category, rate, pickup and impact data.

4.2 Logical Architecture

Layer	Responsibilities	Representative files
Presentation	HTML, Bootstrap layout, forms, tables, status badges, charts	index.php, estimator.php, assets/css/style.css, assets/js/*
Request / page logic	Handle GET/POST, validate fields, route users and render module data	register.php, user/pickup-request.php, admin/pickups.php
Shared security and helpers	Session checks, role checks, CSRF, output encoding, formatting, estimate logic	includes/auth_check.php, includes/admin_check.php, includes/csrf.php, includes/functions.php
Data access	PDO connection, prepared statements and query execution	config/db.php and page-level PDO queries
Persistence	Relational records and foreign-key relationships	config/seed_data.sql; ewaste_db

4.3 Request Processing Flow

A typical request begins when the browser opens a public or protected page. PHP loads configuration and shared helpers. Protected pages invoke the appropriate session/role guard. For a form submission, the server validates the CSRF token, parses and validates user input, performs database work using parameterized statements, and returns a success/error message or redirects to the next page. The browser then renders the response. The database is the persistent source for stored requests and configuration values.

4.4 Database Design

The schema contains six core tables. Users stores account and role information. Collection centres stores location and contact data. Device categories stores unit type, metal-yield coefficients, CO_2-saving coefficients and toxic-diversion coefficients. Metal rates stores benchmark price per gram. Pickup requests stores booking details, theoretical estimates, verification measurements and lifecycle status. Impact logs stores environmental impact values linked to pickup records.

4.5 Entity Relationship Summary

Relationship	Cardinality / rule
users → pickup_requests	One user can create many requests; each request belongs to one user. Deleting a user cascades to their requests in the supplied schema.
collection_centers → pickup_requests	One centre can be associated with many requests; a request may have no centre yet. Deleting a centre sets center_id to NULL.
device_categories → pickup_requests	One category can be used by many requests; deleting a category

	referenced by a request is restricted.
users → metal_rates	An administrator/user record may be referenced as updated_by; deleting that user sets updated_by to NULL.
pickup_requests → impact_logs	A pickup may have an impact log; deleting the pickup cascades to the associated log. The code uses an upsert pattern for recording impact.

4.6 Data Dictionary

Table	Key fields	Purpose
users	id, fullname, email, password, role, city, pincode	Authentication, profile and role information
collection_centers	id, center_name, address, city, state, accepted_items, is_active	Directory and assignment of collection facilities
device_categories	id, category_name, unit_type, *_yield_g, co2_saving_kg, toxic_diverted_kg	Configurable reference coefficients for each e-waste category
metal_rates	metal_name, symbol, rate_per_gram, updated_by, last_updated	Indicative metal prices in INR per gram
pickup_requests	request_code, user_id, category, quantity, est_*, actual_*, status, is_verified	Booking lifecycle and estimate-versus-verification record
impact_logs	pickup_id, co2_saved_kg, toxic_waste_diverted_kg, precious_metals_reclaimed_g	Impact audit values associated with a pickup

4.7 Estimation Logic

For a selected category and positive quantity Q, the server retrieves the active category coefficients. Each estimated metal mass is calculated as the category yield multiplied by Q. The monetary estimate is the sum of each estimated metal mass multiplied by its configured rate per gram. Estimated CO_2 savings and toxic diversion are also category coefficients multiplied by Q. The calculation is a mathematical model; it does not assay the device or guarantee a market payout.

4.8 Formulae

Quantity	Q = user-supplied quantity in the category's configured unit
Metal yield	Estimated metal mass (g) = yield coefficient (g per unit or kg) × Q
Metal value	Estimated value for metal = estimated mass (g) × rate (INR/g)
Total value	Estimated total value = sum of the four individual metal values
CO_2 estimate	Estimated CO_2 saving (kg) = category CO_2 coefficient × Q
Toxic diversion	Estimated toxic diversion (kg) = category toxic-diversion coefficient × Q

4.9 Pickup Lifecycle

The database defines the statuses Pending, Assigned, Out for Pickup, Collected, Completed and Cancelled. A citizen creates a request with a generated code and an initial Pending status. An administrator can assign a centre and progress the status. The verification page records actual weight, actual metal quantities, payout and notes, marks the request verified, and changes its status to Completed. The status history is represented by the current status and timestamps rather than a separate event-by-event audit table in the supplied schema.

4.10 Interface Design

The public interface includes a home page with calls to action and a quick estimator; awareness and reuse pages; a detailed estimator; and a searchable centre directory. The citizen area contains a dashboard, pickup form, history, request detail/print view, impact page and profile page. The administrator area contains a dashboard, pickup management, centre management, metal-rate management, user directory and verification form. The shared header, navigation, footer and stylesheet provide a consistent visual identity.

4.11 Security Design

The source package includes PDO prepared statements, password_hash/password_verify patterns, session regeneration, output sanitization through htmlspecialchars, CSRF token utilities, and user/admin guards. These controls are valuable but do not by themselves establish that the application is production-secure. Deployment should additionally review demo credentials, error-message exposure, session cookie flags, rate limiting, authorization on every action, backup policy and the use of a least-privilege database account.

4.12 Design Limitations

The supplied code is a monolithic PHP application with page-level query logic rather than a centralized service/repository layer. Status transitions are stored as a current enum and can be updated by administrators; the supplied schema does not retain a full status event log. Centre verification and metal coefficients depend on the accuracy of data entered by administrators. These are acceptable simplifications for a college prototype but should be documented before wider deployment.

CHAPTER 5: IMPLEMENTATION AND MODULE DESCRIPTION

5.1 Project Structure

The project folder separates configuration, shared includes, public pages, user pages, admin pages, CSS and JavaScript assets. config/db.php creates the PDO connection. config/constants.php defines application constants and starts a session. includes/functions.php provides formatting, flash messaging, request-code generation, metal-rate retrieval, recovery calculations and global impact aggregation. includes/auth_check.php and includes/admin_check.php protect user and admin routes.

5.2 Public Home Page

index.php loads the configuration, retrieves metal rates and platform statistics, and displays the main calls to action. The page includes a quick estimator preview using device-category values. Where a database is unavailable, the helper functions can return fallback values for some public metrics; such fallback/demo values must not be interpreted as verified real-world totals.

5.3 Awareness and Reuse Modules

awareness.php presents information about hazardous materials associated with electronic equipment and practical disposal guidance. reuse-tips.php presents reuse and refurbishment suggestions and an interactive repair-or-recycle decision aid. These modules serve an educational purpose and help users consider repair or reuse before recycling.

5.4 Metal Recovery Estimator

estimator.php and assets/js/estimator.js provide a device selector, quantity input, live calculation preview and an explanation that results are theoretical. The PHP helper calculateEstimatedRecovery() retrieves category coefficients and current stored rates, calculates each metal mass and value, and returns a structured result. The project records estimates separately from actual values entered after physical verification.

5.5 Collection Centre Locator

centers.php queries active centre records and supports location/keyword filtering. Centre records can contain a name, contact person, telephone, email, address, city, state, postal code, operating hours, accepted materials and coordinates. The README describes one-click map directions. The validity and authorization status of real-world centres depend on the accuracy of the data supplied to the database.

5.6 Registration and Authentication

register.php captures citizen profile information and stores a password hash. login.php validates credentials and routes users based on their role. The shared authentication helpers check the session for protected routes. logout.php destroys the session and redirects the user. Demo login functionality is documented in the README; before a public deployment, demo credentials should be removed or disabled and initial administrator access should be secured.

5.7 Citizen Dashboard and Profile

user/dashboard.php summarizes account activity and recent requests. user/profile.php allows the user to update contact/profile information and password. The dashboard relies on the database records and helper calculations; values should be interpreted according to whether they are estimates or verified results.

5.8 Pickup Request and History

user/pickup-request.php accepts the device category, quantity, preferred date and slot, pickup address, condition and optional notes. It calculates a theoretical recovery preview and stores estimate fields with the request. user/history.php lists requests and supports status-oriented navigation. user/view-request.php presents a detailed printable view and compares theoretical estimates with verified measurements when available.

5.9 Environmental Impact View

user/impact.php presents environmental indicators and Chart.js visualizations, including a metal breakdown and carbon-related values, and offers a printable green-citizen certificate. The values are generated from the stored model coefficients and verification records. They should be described as estimated or recorded values, not as an independent life-cycle assessment.

5.10 Administrator Dashboard

admin/dashboard.php provides an overview of system activity, including users, pickup requests, request status and category-related statistics. Charts are intended to support operational monitoring. The reliability of the metrics depends on the quality and completeness of database records.

5.11 Pickup Management and Verification

admin/pickups.php supports filtering requests, assigning a collection centre and updating status/remarks. admin/verify-pickup.php accepts actual weight, actual gold, silver, copper and aluminium values, payout and verification notes. On successful submission it sets is_verified to 1, marks the request Completed and writes or updates an impact log. This workflow provides a clear separation between the initial estimate and the later values entered by an administrator.

5.12 Centre, Rate and User Administration

admin/centers.php provides centre record management, including adding, editing, deleting and activation controls. admin/metal-rates.php manages benchmark rates per gram. admin/users.php provides a searchable user directory and role-related administration. Administrative actions should be limited to authorized administrators and reviewed carefully in production deployments.

5.13 Calculation Pseudocode

```
INPUT category_id, quantity
IF quantity <= 0: reject request
FETCH active category coefficients
IF category is unavailable: return no estimate
```

```
FOR each metal in [gold, silver, copper, aluminium]:
    estimated_grams[metal] = coefficient[metal] × quantity
    estimated_value[metal] = estimated_grams[metal] × rate_per_gram[metal]
total_value = sum(estimated_value for all metals)
co2_estimate = co2_coefficient × quantity
toxic_diversion_estimate = toxic_coefficient × quantity
RETURN estimates with an explicit theoretical flag
```

5.14 Request-Code Generation

The helper generateRequestCode() creates a human-readable code using a year prefix and a short random hexadecimal suffix. The database enforces uniqueness on request_code. Because the suffix is short, a production implementation should handle duplicate-key collisions by retrying generation and should consider a longer identifier for larger volumes.

5.15 Installation Procedure

- Install XAMPP with Apache and MySQL/MariaDB.
- Copy the extracted project folder into the XAMPP htdocs directory as described in README.md.
- Start Apache and MySQL from the XAMPP Control Panel.
- Open phpMyAdmin and import config/seed_data.sql. Note that this script drops and recreates the project tables; do not import it into a database containing data that must be preserved.
- Confirm the database name and local credentials in config/db.php.
- Open the project's local URL in a browser.
- Use the demo accounts only in a controlled academic environment, and change/remove them before any public deployment.
- Test registration, login, estimator, pickup submission, administrator status updates and verification workflow.

5.16 Implementation Notes

The supplied archive contains PHP pages for the listed public, citizen and administrator modules, shared assets and a seed SQL file. This report is based on source inspection. The application was not executed in a live browser/database environment during report generation; therefore, statements about intended functionality should not be read as a claim that every workflow has passed runtime testing.

CHAPTER 6: TESTING AND RESULTS DISCUSSION

6.1 Testing Approach

Testing should combine module-level checks, integration checks, security checks and responsive-interface checks. The project blueprint lists representative scenarios for authentication, access control, CSRF/XSS, estimator calculations, pickup booking, actual-versus-estimated verification, responsive layouts and centre filtering. Since no runtime test logs or screenshots were included in the supplied package, the cases below are presented as a test plan and acceptance criteria rather than fabricated pass results.

6.2 Test Cases

ID	Module	Test / input	Expected result	Evidence status
TC-01	Registration	Submit valid new user details	User is stored with hashed password and default citizen role	To execute
TC-02	Login	Sign in with valid credentials	Session is established and role-based redirect occurs	To execute
TC-03	Access control	Open admin page as guest/citizen	Request is blocked or redirected by admin guard	To execute
TC-04	CSRF	Submit a POST form without a valid token	Request is rejected with a security message	To execute
TC-05	XSS output	Store/render text containing HTML/script characters	Output is encoded and script does not execute	To execute
TC-06	Estimator	Select category and positive quantity	Metal mass/value and impact estimates are calculated from stored coefficients/rates	To execute
TC-07	Estimator validation	Enter zero, negative or invalid quantity	Invalid quantity is rejected or no estimate is returned	To execute
TC-08	Pickup booking	Submit a valid pickup request	Unique code, request row and Pending status are created	To execute
TC-09	Pickup dispatch	Assign active centre and change status	Request is updated and appears in the corresponding admin filter	To execute
TC-10	Verification	Enter actual weights and submit verification	Actual values are stored, is_verified is set, status becomes Completed, impact log is written/updated	To execute
TC-11	Centre locator	Filter by city or keyword	Only matching active centres are shown	To execute
TC-12	Responsive layout	Test 375px, 768px and 1200px widths	Navigation, forms, cards and tables remain usable	To execute
TC-13	Rate management	Update a metal rate as admin	Subsequent estimates use the updated rate	To execute
TC-14	Request privacy	Citizen attempts to view another citizen's request	Server verifies ownership or denies	To execute

			access	

6.3 Calculation Validation

For a category coefficient y and quantity Q, the expected mass is y × Q. For example, if a category coefficient is 0.024 grams of gold per unit and the quantity is five units, the mathematical output is 0.120 grams of gold. This example demonstrates multiplication only; it does not validate the scientific accuracy of the coefficient. A complete test should compare the server-side calculation with an independently calculated expected result and verify that the same rate values are used by the UI and server.

6.4 Security Validation

Security testing should include invalid credentials, session renewal after login, direct URL access to protected pages, CSRF token omission, output containing special HTML characters, malformed identifiers and attempts to alter request IDs or roles. It should also confirm that users can view only their own request records. Source presence of a guard or sanitizer is evidence of an intended control, but only runtime testing can confirm that the control covers every relevant route and action.

6.5 Results Supported by Source Inspection

The archive contains the major public, citizen and administrator PHP pages described in this report, a MySQL schema/seed script, shared authentication and CSRF helpers, a common calculation helper, JavaScript assets and a setup README. The source includes separate estimated and actual metal fields, a verification flag and a pickup status field. The verification page updates these fields and records an impact log. These observations support a description of the implemented structure, but they are not substitutes for a test run against a configured database.

6.6 Known Risks and Findings to Verify

- Run the complete setup and test all routes with the supplied seed database.
- Confirm that all protected request-detail queries enforce citizen ownership, not only authentication.
- Review every POST action for CSRF validation and every user-controlled output for context-appropriate encoding.
- Check that numeric inputs are bounded and reject negative weights, values and quantities where not meaningful.
- Ensure the impact log upsert is supported by an appropriate unique key on pickup_id; confirm the supplied SQL schema actually defines it.
- Check that all status values in filters, forms and schema use identical spelling and capitalization.
- Review fallback dashboard counters so they are clearly marked as demo data and never presented as real impact.
- Validate all sample collection-centre details before representing them as authorized or certified facilities.

- Remove or disable public demo-login buttons and default demo accounts before production deployment.
- Run accessibility checks for labels, keyboard navigation, colour contrast and chart alternatives.

6.7 Discussion

The system's strongest conceptual contribution is the integration of awareness, estimation, pickup logistics and a verification record in one small web application. Its data model allows an estimate to be retained at booking time while actual values are recorded later. This can support transparency and educational discussion. However, the scientific validity of estimates depends on the coefficient dataset; operational usefulness depends on real, verified collection-centre information; and security depends on runtime coverage and deployment practices. These areas should be treated as ongoing validation tasks.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

E-Waste to Best is a modular PHP/MySQL web application designed to support responsible e-waste handling through public awareness, repair/reuse guidance, theoretical recovery estimation, centre discovery, citizen pickup requests, administrative coordination and post-collection verification. The supplied source package shows a structured implementation with reusable helpers, role guards, CSRF utilities, PDO-based database access and separate data fields for estimates and verified measurements. The project demonstrates how a web platform can make the e-waste collection process more visible and understandable.

7.2 Achievements

- Organized the application into public, citizen and administrator modules.
- Designed a relational schema with six core tables and foreign-key relationships.
- Implemented a calculation helper for four metals and indicative environmental metrics.
- Included a pickup lifecycle with request codes and administrator status updates.
- Provided an interface for recording actual post-collection values separately from estimates.
- Included educational pages and a reuse-first perspective alongside recycling logistics.
- Documented local installation through XAMPP and a SQL seed script.

7.3 Limitations

- Metal-yield and environmental coefficients are configurable model inputs and require traceable, category-specific scientific validation.
- Actual recovery data depends on physical measurement and correct entry by authorized personnel.
- The centre directory and its certification claims depend on the accuracy of the records entered.
- The supplied package does not prove runtime test completion or independent security review.
- The current schema stores the current request status rather than a full immutable status-event history.
- The project is designed for local academic demonstration and does not document production hosting, backup/restore, monitoring or privacy governance in depth.

7.4 Future Enhancements

- Integrate a verified directory or official API for authorized e-waste collection and recycling facilities.
- Add a full audit trail for status changes, assignments, measurement edits and administrator actions.
- Improve the scientific model with citations, device-specific uncertainty ranges, and versioned coefficient datasets.
- Add role-specific access for collection-centre staff and laboratory personnel.
- Introduce notifications by email/SMS for booking confirmations and status changes.
- Add map-based centre discovery with validated coordinates and accessibility details.
- Provide downloadable PDF receipts and certificates generated from verified records.

- Implement automated backup, structured logging, rate limiting and stronger session-cookie settings.
- Create automated unit and integration tests, CI checks and an accessible UI test suite.
- Consider a production framework and service/repository layer if the application grows.

7.5 Final Remarks

Digital tools cannot replace authorized handling, safe dismantling or laboratory assays. Their value is in making information, coordination and recordkeeping easier. E-Waste to Best provides a useful academic prototype for demonstrating those ideas, provided its estimates are clearly labelled and its data is validated before real-world use.

REFERENCES

1. Project source package: “E-Waste to Best — Smart E-Waste Recycling & Metal Recovery System,” including README.md, PHP source files, JavaScript/CSS assets and config/seed_data.sql, supplied for this report.
2. PHP Documentation. PHP Manual: password_hash(), password_verify(), PDO and session handling. https://www.php.net/docs.php
3. MySQL Documentation. MySQL Reference Manual: InnoDB, foreign-key constraints and character sets. https://dev.mysql.com/doc/
4. Bootstrap Documentation. Bootstrap 5.3 documentation. https://getbootstrap.com/docs/5.3/
5. Chart.js Documentation. Chart.js user guide. https://www.chartjs.org/docs/
6. OWASP Foundation. SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/
7. OWASP Foundation. Cross Site Scripting Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/
8. OWASP Foundation. Cross-Site Request Forgery Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/
9. Apache Friends. XAMPP documentation and downloads. https://www.apachefriends.org/
10. U.S. Geological Survey. Circular 1400, Critical Mineral Resources of the United States (project README references USGS material as background; verify the exact publication and coefficient source before relying on individual yield values).
11. U.S. Environmental Protection Agency. Electronics donation and recycling resources (consult current official guidance for jurisdiction-specific disposal requirements). https://www.epa.gov/recycle/electronics-donation-and-recycling
12. Global E-waste Monitor. Global e-waste reporting and data resources. https://ewastemonitor.info/

APPENDIX A: INSTALLATION CHECKLIST

- ☐ Extract the project archive and preserve the folder structure.
- ☐ Start Apache and MySQL/MariaDB in XAMPP.
- ☐ Import config/seed_data.sql into a development database. Warning: the SQL script drops existing project tables before recreating them.
- ☐ Confirm the database name and local credentials in config/db.php.
- ☐ Open the application locally and inspect the home page, awareness, reuse tips, estimator and centre locator.
- ☐ Register a test citizen account and verify the password is stored as a hash.
- ☐ Create a pickup request and confirm that a request code and Pending status are stored.
- ☐ Sign in as administrator, assign a centre, progress the status and record verification values.
- ☐ Verify that actual values remain separate from theoretical estimates.
- ☐ Capture genuine screenshots and record test outcomes before final institutional submission.

APPENDIX B: PROJECT FILE MAP

Path / file	Role
config/db.php	PDO database connection
config/constants.php	Application constants, base URL and session start
config/seed_data.sql	Database schema and sample data setup
includes/functions.php	Sanitization, formatting, estimates, request codes, impact statistics
includes/auth_check.php	Citizen session guard
includes/admin_check.php	Administrator role guard
includes/csrf.php	CSRF token generation and validation
assets/css/style.css	Shared visual styles
assets/js/main.js	General interface behaviour
assets/js/estimator.js	Estimator interaction logic
index.php, awareness.php, reuse-tips.php, estimator.php, centers.php	Public-facing modules
register.php, login.php, logout.php	Authentication
user/*.php	Citizen dashboard, requests, history, impact and profile
admin/*.php	Admin dashboard, pickup management, centres, rates, users and verification

APPENDIX C: PROJECT SCREENSHOTS AND EXPLANATION

Visual walkthrough of the supplied E-Waste to Best interface

The following figures are screenshots supplied with the project materials. They illustrate the interface design and the intended user journey. The figures show interface content; they do not, by themselves, prove that all backend operations or environmental calculations have been independently tested.

C.1 Home Page and Quick Metal Estimator

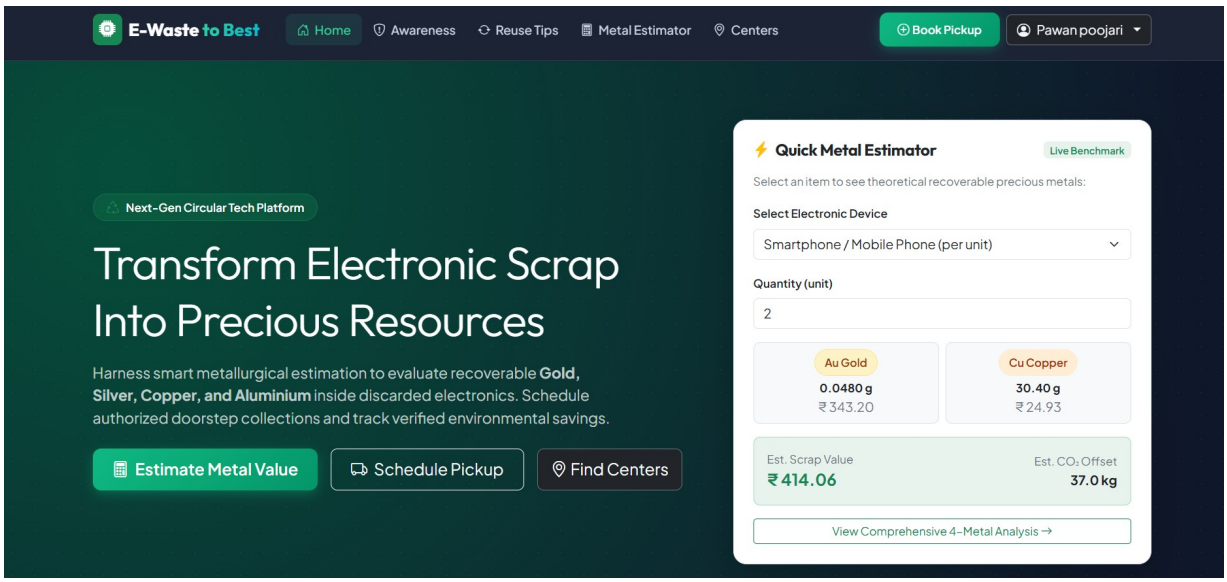


Figure C.1: Home page with navigation and quick metal estimator.

The top navigation gives access to Home, Awareness, Reuse Tips, Metal Estimator and Centres. The main section explains the purpose of the platform in simple language. The quick estimator panel lets a user choose an electronic-device category and enter a quantity. The interface then displays estimated metal quantities, indicative scrap value and an estimated CO_2 offset. These values are model-based estimates using configured per-item coefficients and benchmark rates. They are not a guaranteed sale price or a laboratory measurement.

C.2 Environmental Impact Dashboard

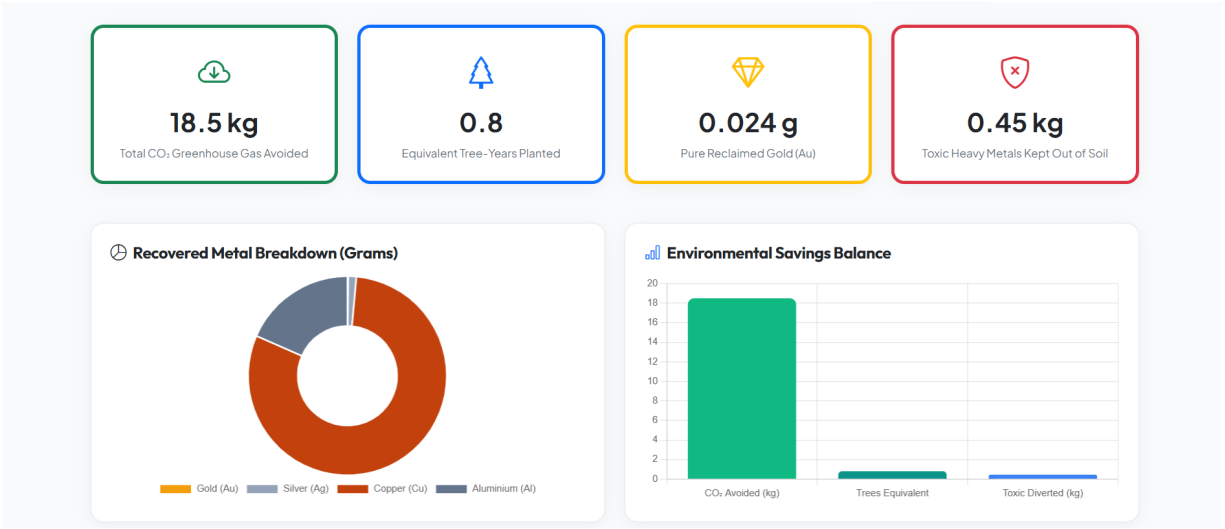


Figure C.2: Dashboard cards and charts for environmental indicators and recovered-metal breakdown.

The four summary cards present CO_2 greenhouse-gas emissions avoided, equivalent tree-years, estimated reclaimed gold and toxic heavy metals diverted from soil. Below them, a doughnut chart compares the estimated quantities of gold, silver, copper and aluminium, while a bar chart compares selected environmental indicators. Charts make the figures easier to compare at a glance. For a real deployment, the page should clearly identify whether each value comes from an estimate, an administrator-entered verified measurement, or sample/demo data. Tree-equivalent values are especially dependent on the conversion factor used.

C.3 Circular Recovery Process

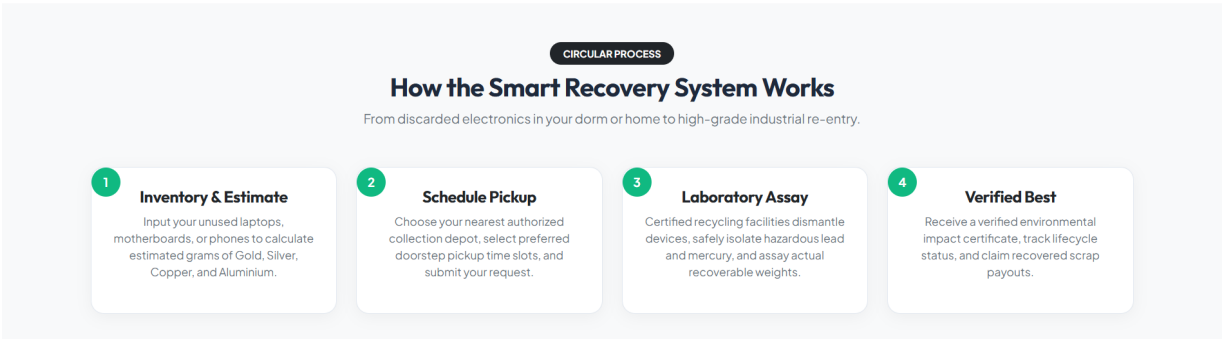


Figure C.3: Four-stage process from inventory and estimation to verified recovery.

Stage 1 is Inventory & Estimate: the user selects an electronic item and reviews possible recoverable materials. Stage 2 is Schedule Pickup: the user chooses a collection centre and a preferred time slot, then submits a request. Stage 3 is Laboratory Assay: the diagram describes authorized facilities dismantling devices and measuring recovered materials safely. Stage 4 is Verified Best: the platform is intended to record verified results and show lifecycle status. The laboratory and payout steps depend on real-world partners and verified records; the web application alone does not physically recycle devices or perform chemical assays.

C.4 Reading the Figures Correctly

The screenshots communicate the project concept and the planned user experience. Any displayed number should be treated as an estimate unless the system records a measured result with a clear verification status. For academic demonstrations, test data should be labelled as sample data. Environmental savings depend on assumptions about the device type, recovery process and baseline disposal method, so the calculation factors should be documented and validated before the numbers are used for public claims.

APPENDIX D: SELECTED SOURCE CODE AND EXPLANATION

Code excerpts are from the supplied project ZIP; comments explain their role

This appendix includes short excerpts from the actual project files to show how important tasks are implemented. The excerpts are shortened for readability. The complete source files remain the authoritative implementation.

D.1 PHP: Input Safety for HTML Output (includes/functions.php)

```
function sanitize($input) {
    if (is_array($input)) {
        return array_map('sanitize', $input);
    }
    return htmlspecialchars(trim((string)$input), ENT_QUOTES, 'UTF-8');
}
```

This helper trims surrounding spaces and converts special HTML characters into safe text before output. It helps reduce cross-site scripting risk when displaying user-controlled values. It is important to use context-appropriate output encoding consistently throughout the application.

D.2 PHP: Theoretical Metal Recovery Calculation (includes/functions.php)

```
function calculateEstimatedRecovery($pdo, $categoryId, $quantity) {
    $qty = (float)$quantity;
    if ($qty <= 0) {
        return null;
    }

    if (!$pdo) {
        return null;
    }

    try {
        $stmt = $pdo->prepare("SELECT * FROM device_categories WHERE id = ? AND is_active = 1");
        $stmt->execute([$categoryId]);
        $cat = $stmt->fetch();
        if (!$cat) return null;

        $copper_g = (float)$cat['copper_yield_g'] * $qty;
        $aluminium_g = (float)$cat['aluminium_yield_g'] * $qty;
        $silver_g = (float)$cat['silver_yield_g'] * $qty;
        $gold_g = (float)$cat['gold_yield_g'] * $qty;
        $co2_kg = (float)$cat['co2_saving_kg'] * $qty;
        $toxic_kg = (float)$cat['toxic_diverted_kg'] * $qty;

        $rates = getMetalRates($pdo);
        $goldRate = $rates['Gold']['rate'] ?? 7150.00;
    // Remaining value assembly and return fields are defined in the source file.
```

The function checks that quantity is positive, looks up an active device category using a prepared statement, and multiplies each category yield by the requested quantity. It also calculates environmental indicators and uses configured metal rates to estimate value. Because the coefficients are assumptions, the result must be labelled theoretical and should not be presented as a certified measure

D.3 JavaScript: Live Estimator Calculation (assets/js/estimator.js)

```
const totalCopperG = copperYieldPerUnit * qty;
const totalAluminiumG = aluminiumYieldPerUnit * qty;
const totalSilverG = silverYieldPerUnit * qty;
const totalGoldG = goldYieldPerUnit * qty;

const valCopper = totalCopperG * rates.Copper;
const valAluminium = totalAluminiumG * rates.Aluminium;
const valSilver = totalSilverG * rates.Silver;
const valGold = totalGoldG * rates.Gold;
const totalVal = valCopper + valAluminium + valSilver + valGold;
```

This JavaScript code updates the estimator quickly when a user changes the device type or quantity. The metal mass is calculated by multiplying the per-unit yield by the quantity. The estimated value is calculated by multiplying each metal mass by its benchmark rate and adding the results. Browser-side values improve the user experience, but the server must independently validate and calculate any value stored in the database.

D.4 PHP: CSRF Token Validation (includes/csrf.php)

```
function validateCsrfToken($token = null) {
  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $submittedToken = $token ?? ($_POST['csrf_token'] ?? '');
    if (empty($_SESSION['csrf_token']) ||
      !hash_equals($_SESSION['csrf_token'], $submittedToken)) {
      return false;
    }
  }
  return true;
}
```

Cross-Site Request Forgery (CSRF) protection checks that a form submission includes a token associated with the user session. The token should be added to every state-changing form and validated before processing the action. The presence of a helper does not prove that every form uses it, so all POST routes should be reviewed during testing.

D.5 PHP: PDO Database Connection (config/db.php)

```
$pdo = new PDO(
  "mysql:host={$db_host};dbname={$db_name};charset=utf8mb4",
  $db_user,
  $db_pass,
  [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false,
  ]
);
```

PDO provides a standard way to connect PHP to MySQL or MariaDB. Exception-based error handling helps the application detect database failures, associative fetches return rows using column names, and native prepared statements are preferred for parameterized queries.

Production deployment should use a dedicated database account with limited permissions rather than a default local development account.

D.6 Application Constants and Session Startup

Source file: config/constants.php

```
<?php
/**
 * Application Constants & Configuration
 * Project: E-Waste to Best - Smart E-Waste Recycling & Metal Recovery System
 */

// Application Info
define('APP_NAME', 'E-Waste to Best');
define('APP_TAGLINE', 'Smart E-Waste Recycling & Metal Recovery System');
define('APP_VERSION', '1.0.0');

// Currency & Units
define('CURRENCY_SYMBOL', '₹');
define('CURRENCY_CODE', 'INR');

// Base Paths & URLs
if (!defined('BASE_URL')) {
    // Automatically detect protocol and host
    $protocol = (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off' || (isset($_SERVER['SERVER_PORT']) && $_SERVER['SERVER_PORT'] ==
443)) ? "https://" : "http://";
    $host = $_SERVER['HTTP_HOST'] ?? 'localhost';

    // Determine the base path relative to web root
    $scriptName = str_replace('\\', '/', $_SERVER['SCRIPT_NAME'] ?? '');
    $baseFolder = '';

    // Find the root folder name 'Waste to best' or similar in script path
    $parts = explode('/', trim($scriptName, '/'));
    if (!empty($parts)) {
        // If the first part is a PHP file (e.g. index.php), the site is running at web root
        if (substr($parts[0], -4) === '.php') {
            $baseFolder = '';
        } else {
            $baseFolder = '/' . $parts[0];
        }
    }

    define('BASE_URL', $protocol . $host . $baseFolder);
}

// Session configuration
if (session_status() === PHP_SESSION_NONE) {
    session_start();
```

Explanation and learning notes

This configuration file centralizes the application name, version, Indian currency settings, base URL detection, and session startup. Keeping these values in one place reduces duplication. The excerpt is from the supplied project; database credentials remain in config/db.php.

D.7 PDO Database Connection and Error Mode

Source file: config/db.php

```
<?php
/**
 * Database Connection using PDO
 * Project: E-Waste to Best
 */

require_once __DIR__ . '/constants.php';

$db_host = 'localhost';
$db_name = 'ewaste_db';
$db_user = 'root';
$db_pass = ''; // Default XAMPP password is empty

try {
    $pdo = new PDO("mysql:host={$db_host};dbname={$db_name};charset=utf8mb4", $db_user, $db_pass, [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ]);
} catch (PDOException $e) {
    // If running in development and DB doesn't exist yet, show friendly setup instructions
    $errorMsg = $e->getMessage();

    // Check if error is unknown database
    if (strpos($errorMsg, 'Unknown database') !== false) {
        $dbSetupNotice = true;
    }

    // Provide a helper function to check DB availability
    function isDbConnected() {
        return false;
    }

    $pdo = null;
}

if (!function_exists('isDbConnected')) {
    function isDbConnected() {
        global $pdo;
        return $pdo !== null;
    }
}
```

Explanation and learning notes

PDO creates the connection used by the PHP pages. Exception mode allows database failures to be caught, associative fetch mode returns rows by column name, and native prepared statements are preferred. For deployment, database credentials should be stored outside publicly accessible source files and changed from development defaults.

D.8 Flash Messages and One-Time User Feedback

Source file: includes/functions.php

```
    }
    return htmlspecialchars(trim((string)$input), ENT_QUOTES, 'UTF-8');
}

/**
 * Set flash message to session
 */
function setFlash($type, $message) {
    $_SESSION['flash'] = [
        'type' => $type, // 'success', 'danger', 'warning', 'info'
        'message' => $message
    ];
}

/**
 * Get and clear flash message
 */
function getFlash() {
    if (isset($_SESSION['flash'])) {
        $flash = $_SESSION['flash'];
        unset($_SESSION['flash']);
        return $flash;
    }
    return null;
}

/**
 * Render Bootstrap Alert for Flash message if present
 */
function renderFlash() {
    $flash = getFlash();
    if ($flash) {
        $alertType = $flash['type'] === 'error' ? 'danger' : $flash['type'];
        $icon = match($alertType) {
```

Explanation and learning notes

The flash-message helpers store a message in the session and clear it after retrieval. This supports short confirmations such as a successful booking or a warning. The display helper uses the project's Bootstrap alert styling; output escaping is important whenever content may contain user-controlled text.

D.9 Formatting Currency and Recovered Metal Mass

Source file: includes/functions.php

```
        'success' => 'bi-check-circle-fill',
        'danger' => 'bi-exclamation-triangle-fill',
        'warning' => 'bi-exclamation-circle-fill',
        default => 'bi-info-circle-fill'
    };
    echo '
    <div class="alert alert-' . sanitize($alertType) . ' alert-dismissible fade show d-flex align-items-center shadow-sm" role="alert">
        <i class="bi ' . $icon . ' me-2 fs-5"></i>
        <div>' . $flash['message'] . '</div>
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
    </div>';
  }
}

/**
 * Format currency with configured symbol
 */
function formatCurrency($amount) {
  return CURRENCY_SYMBOL . ' ' . number_format((float)$amount, 2);
}
```

Explanation and learning notes

Formatting helpers keep the interface consistent. Currency is shown with the configured rupee symbol and two decimal places. Metal mass is shown in grams, or converted to kilograms for larger values. Formatting changes presentation only; it should not be confused with the underlying calculation.

D.10 Loading Current Metal Benchmark Rates

Source file: includes/functions.php

```
 * Format gram values gracefully
 */
function formatGrams($grams, $decimals = 3) {
    $g = (float)$grams;
    if ($g >= 1000) {
        return number_format($g / 1000, 2) . ' kg';
    }
    return number_format($g, $decimals) . ' g';
}

/**
 * Generate human-friendly Unique Request Code (e.g. EWB-2026-4819)
 */
function generateRequestCode() {
    return 'EWB-' . date('Y') . '-' . strtoupper(substr(bin2hex(random_bytes(3)), 0, 4));
}

/**
 * Get all current benchmark metal rates
 */
function getMetalRates($pdo) {
    if (!$pdo) {
        return [
            'Gold' => ['symbol' => 'Au', 'rate' => 7150.00],
            'Silver' => ['symbol' => 'Ag', 'rate' => 88.50],
```

Explanation and learning notes

The rate helper reads the metal name, chemical symbol, and rate per gram from the database. If a database connection is not available, the current source contains fallback benchmark values. These are illustrative application rates and must not be described as guaranteed live market prices.

D.11 Backend Theoretical Metal Recovery Calculation

Source file: includes/functions.php

```
        'Aluminium' => ['symbol' => 'Al', 'rate' => 0.24]
    ];
  }
  try {
    $stmt = $pdo->query("SELECT metal_name, symbol, rate_per_gram FROM metal_rates");
    $rates = [];
    while ($row = $stmt->fetch()) {
      $rates[$row['metal_name']] = [
        'symbol' => $row['symbol'],
        'rate' => (float)$row['rate_per_gram']
      ];
    }
    return $rates;
  } catch (PDOException $e) {
    return [];
  }
}

/**
 * Theoretical recovery calculation based on category and quantity
 */
function calculateEstimatedRecovery($pdo, $categoryId, $quantity) {
  $qty = (float)$quantity;
  if ($qty <= 0) {
    return null;
  }

  if (!$pdo) {
    return null;
  }

  try {
    $stmt = $pdo->prepare("SELECT * FROM device_categories WHERE id = ? AND is_active = 1");
    $stmt->execute([$categoryId]);
    $cat = $stmt->fetch();
    if (!$cat) return null;

    $copper_g = (float)$cat['copper_yield_g'] * $qty;
    $aluminium_g = (float)$cat['aluminium_yield_g'] * $qty;
    $silver_g = (float)$cat['silver_yield_g'] * $qty;
    $gold_g = (float)$cat['gold_yield_g'] * $qty;
    $co2_kg = (float)$cat['co2_saving_kg'] * $qty;
    $toxic_kg = (float)$cat['toxic_diverted_kg'] * $qty;

    $rates = getMetalRates($pdo);
    $goldRate = $rates['Gold']['rate'] ?? 7150.00;
    $silverRate = $rates['Silver']['rate'] ?? 88.50;
    $copperRate = $rates['Copper']['rate'] ?? 0.82;
    $aluminiumRate = $rates['Aluminium']['rate'] ?? 0.24;

    $goldValue = $gold_g * $goldRate;
    $silverValue = $silver_g * $silverRate;
    $copperValue = $copper_g * $copperRate;
    $aluminiumValue = $aluminium_g * $aluminiumRate;
    $totalValue = $goldValue + $silverValue + $copperValue + $aluminiumValue;
```

Explanation and learning notes

This is the core server-side estimator. It rejects non-positive quantity, looks up an active device category with a prepared statement, multiplies the category yield by quantity, and calculates an estimated value using the configured rates. The returned flag is_theoretical makes the estimate status explicit; actual recovery requires physical verification.

D.12 Unique Pickup Request Code

Source file: includes/functions.php

```
/**
 * Format currency with configured symbol
 */
function formatCurrency($amount) {
    return CURRENCY_SYMBOL . ' ' . number_format((float)$amount, 2);
}
```

Explanation and learning notes

The project generates a human-readable request identifier using the prefix EWB, the current year, and random bytes. The code helps users and administrators refer to a booking without using a database row number. Random identifiers should still be checked for uniqueness at the database level if collision handling is required.

D.13 CSRF Token Generation and Validation

Source file: includes/csrf.php

```
<?php
/**
 * CSRF Protection Helpers
 * Project: E-Waste to Best
 */

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

/**
 * Generate CSRF token if not already existing
 */
function generateCsrfToken() {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

/**
 * Render hidden HTML input field with CSRF token
 */
function csrfField() {
    $token = generateCsrfToken();
    return '<input type="hidden" name="csrf_token" value="' . htmlspecialchars($token, ENT_QUOTES, 'UTF-8') . '">';
}

/**
 * Validate CSRF token on POST requests
 */
function validateCsrfToken($token = null) {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $submittedToken = $token ?? ($_POST['csrf_token'] ?? '');
        if (empty($_SESSION['csrf_token']) || !hash_equals($_SESSION['csrf_token'], $submittedToken)) {
            return false;
        }
    }
```

Explanation and learning notes

A CSRF token helps ensure that a state-changing form was submitted from a session that received the form. The token is stored in the session, placed in a hidden field, and compared with hash_equals during POST validation. This is one layer of protection and should be used together with authentication and server-side validation.

D.14 User Authentication Guard

Source file: includes/auth_check.php

```
<?php
/**
 * User Authentication Guard
 * Project: E-Waste to Best
 */

require_once __DIR__ . '/../config/constants.php';
require_once __DIR__ . '/functions.php';

if (empty($_SESSION['user_id'])) {
    setFlash('warning', 'Please sign in to access your portal.');
    header('Location: ' . BASE_URL . '/login.php');
    exit;
}
```

Explanation and learning notes

This guard checks whether a user ID exists in the session. If the session is not authenticated, the page records a warning and redirects the visitor to the login screen. It protects pages that are intended only for signed-in users; each protected page must include the guard before producing page output.

D.15 Administrator Role Guard

Source file: includes/admin_check.php

```
<?php
/**
 * Admin Access Guard
 * Project: E-Waste to Best
 */

require_once __DIR__ . '/../config/constants.php';
require_once __DIR__ . '/functions.php';

if (empty($_SESSION['user_id'])) {
    setFlash('warning', 'Administrative access requires authentication.');
    header('Location: ' . BASE_URL . '/login.php');
    exit;
}

if (empty($_SESSION['user_role']) || $_SESSION['user_role'] !== 'admin') {
    setFlash('danger', 'Access denied. You do not possess administrator credentials.');
    header('Location: ' . BASE_URL . '/user/dashboard.php');
    exit;
}
```

Explanation and learning notes

Administrator pages require both an authenticated session and the admin role. The second check matters because a normal registered user must not receive administrative privileges merely by signing in. Authorization should be enforced on the server, not only by hiding navigation links.

D.16 Registration Validation and Duplicate Email Check

Source file: register.php

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  if (!validateCsrfToken()) {
    $error = 'Security validation failed (Invalid CSRF Token). Please refresh and try again.';
  } else {
    $fullname = trim($_POST['fullname'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $phone = trim($_POST['phone'] ?? '');
    $city = trim($_POST['city'] ?? '');
    $pincode = trim($_POST['pincode'] ?? '');
    $address = trim($_POST['address'] ?? '');
    $password = $_POST['password'] ?? '';
    $confirmPassword = $_POST['confirm_password'] ?? '';

    // Validation rules
    if (empty($fullname) || empty($email) || empty($phone) || empty($city) || empty($pincode) || empty($address) || empty($password)) {
      $error = 'All fields are required to register your citizen recycling profile.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
      $error = 'Please enter a valid email address.';
    } elseif (strlen($password) < 6) {
      $error = 'Password must be at least 6 characters long.';
    } elseif ($password !== $confirmPassword) {
      $error = 'Passwords do not match. Please verify both password entries.';
    } elseif (!$pdo) {
      $error = 'Database connection is unavailable. Please check your MySQL setup.';
    } else {
      try {
        // Check if email already registered
        $checkStmt = $pdo->prepare("SELECT id FROM users WHERE email = ? LIMIT 1");
        $checkStmt->execute([$email]);
        if ($checkStmt->fetch()) {
          $error = 'An account with this email address already exists. Please sign in instead.';
        } else {
          // Hash password securely using standard algorithm
          $hashedPassword = password_hash($password, PASSWORD_DEFAULT);

          $insertStmt = $pdo->prepare("
            INSERT INTO users (fullname, email, password, phone, address, city, pincode, role)
            VALUES (?, ?, ?, ?, ?, ?, ?, 'user')
          ");
          $insertStmt->execute([
            $fullname,
            $email,
            $hashedPassword,
            $phone,
            $address,
            $city,
            $pincode
          ]);

          setFlash('success', 'Registration successful! You may now sign in with your credentials.');
          header('Location: ' . BASE_URL . '/login.php');
          exit;
        }
      } catch (PDOException $e) {
        $error = 'Registration failed: ' . $e->getMessage();
```

Explanation and learning notes

Registration reads submitted fields, checks required values, validates email format, compares password confirmation, and checks whether the email is already registered. It uses a prepared statement and stores a password hash rather than plain text. The current source's minimum password length is six characters; stronger password policy may be considered separately without changing the current system in this report.

D.17 Login Verification and Session Regeneration

Source file: login.php

```
$error = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!validateCsrfToken()) {
        $error = 'Security validation failed (Invalid CSRF Token). Please refresh and try again.';
    } else {
        $email = trim($_POST['email'] ?? '');
        $password = $_POST['password'] ?? '';

        if (empty($email) || empty($password)) {
            $error = 'Please enter both your email address and password.';
        } elseif (!$pdo) {
            $error = 'Database connection is unavailable. Please check your MySQL setup in XAMPP.';
        } else {
            try {
                $stmt = $pdo->prepare("SELECT id, fullname, email, password, role FROM users WHERE email = ? LIMIT 1");
                $stmt->execute([$email]);
                $user = $stmt->fetch();

                if ($user && password_verify($password, $user['password'])) {
                    // Password matches! Protect against session fixation
                    session_regenerate_id(true);

                    $_SESSION['user_id'] = $user['id'];
                    $_SESSION['user_name'] = $user['fullname'];
                    $_SESSION['user_email'] = $user['email'];
                    $_SESSION['user_role'] = $user['role'];

                    setFlash('success', 'Welcome back, ' . sanitize($user['fullname']) . '!');

                    if ($user['role'] === 'admin') {
                        header('Location: ' . BASE_URL . '/admin/dashboard.php');
                    } else {
                        header('Location: ' . BASE_URL . '/user/dashboard.php');
                    }
                    exit;
                } else {
                    $error = 'Invalid email address or password. Please verify your credentials.';
                }
            } catch (PDOException $e) {
                $error = 'A system error occurred during authentication: ' . $e->getMessage();
            }
        }
```

Explanation and learning notes

The login flow fetches the account by email, verifies the entered password against its stored hash, and regenerates the session ID after successful authentication. The role then determines whether the user is redirected to the administrator dashboard or the citizen dashboard. The redirect and exit prevent the login page from continuing to render after authentication.

D.18 Profile Update and Password Change

Source file: user/profile.php

```
// Handle Profile Information Update
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!validateCsrfToken()) {
        $error = 'Security check failed. Please refresh and try again.';
    } else {
        $action = $_POST['action'] ?? '';

        if ($action === 'update_info') {
            $fullname = trim($_POST['fullname'] ?? '');
            $phone = trim($_POST['phone'] ?? '');
            $address = trim($_POST['address'] ?? '');
            $city = trim($_POST['city'] ?? '');
            $pincode = trim($_POST['pincode'] ?? '');

            if (empty($fullname) || empty($phone) || empty($address) || empty($city) || empty($pincode)) {
                $error = 'All profile fields are required.';
            } else {
                try {
                    $updateStmt = $pdo->prepare("
                        UPDATE users
                        SET fullname = ?, phone = ?, address = ?, city = ?, pincode = ?
                        WHERE id = ?
                    ");
                    $updateStmt->execute([$fullname, $phone, $address, $city, $pincode, $userId]);

                    $_SESSION['user_name'] = $fullname;
                    setFlash('success', 'Profile information updated successfully.');
                    header('Location: ' . BASE_URL . '/user/profile.php');
                    exit;
                } catch (PDOException $e) {
                    $error = 'Failed to update profile: ' . $e->getMessage();
                }
            }
        } elseif ($action === 'change_password') {
            $currentPassword = $_POST['current_password'] ?? '';
            $newPassword = $_POST['new_password'] ?? '';
            $confirmPassword = $_POST['confirm_password'] ?? '';

            if (empty($currentPassword) || empty($newPassword) || empty($confirmPassword)) {
                $error = 'Please fill out all password fields.';
            } elseif (!password_verify($currentPassword, $user['password'])) {
                $error = 'Current password is incorrect.';
            } elseif (strlen($newPassword) < 6) {
                $error = 'New password must be at least 6 characters.';
            } elseif ($newPassword !== $confirmPassword) {
                $error = 'New passwords do not match.';
            } else {
                try {
                    $hashed = password_hash($newPassword, PASSWORD_DEFAULT);
                    $pwdStmt = $pdo->prepare("UPDATE users SET password = ? WHERE id = ?");
                    $pwdStmt->execute([$hashed, $userId]);

                    setFlash('success', 'Your password has been securely updated.');
                    header('Location: ' . BASE_URL . '/user/profile.php');
                    exit;
                } catch (PDOException $e) {
                    $error = 'Password update failed: ' . $e->getMessage();
                }
```

Explanation and learning notes

The profile module separates profile information updates from password changes. The password branch first checks the current password, validates the new password and confirmation, hashes the new password, and then updates the account. This excerpt demonstrates how account changes are handled through POST requests and prepared statements.

D.19 Pickup Form Validation and Estimation

Source file: user/pickup-request.php

```
// Fetch metal rates for client calculation
$metalRates = getMetalRates($pdo);

// Handle Form Submission
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!validateCsrfToken()) {
        $error = 'Security check failed (CSRF token invalid). Please refresh and submit again.';
    } else {
        $categoryId = (int)($_POST['device_category_id'] ?? 0);
        $quantity = (float)($_POST['quantity'] ?? 0);
        $itemCondition = $_POST['item_condition'] ?? 'Dead / Scrap';
        $centerId = !empty($_POST['center_id']) ? (int)$_POST['center_id'] : null;
        $pickupAddress = trim($_POST['pickup_address'] ?? '');
        $pickupCity = trim($_POST['pickup_city'] ?? '');
        $pickupPincode = trim($_POST['pickup_pincode'] ?? '');
        $preferredDate = $_POST['preferred_date'] ?? '';
        $preferredSlot = $_POST['preferred_slot'] ?? '';
        $specialNotes = trim($_POST['special_notes'] ?? '');

        // Validation
        if ($categoryId <= 0 || $quantity <= 0) {
            $error = 'Please select a valid device category and enter a quantity greater than zero.';
        } elseif (empty($pickupAddress) || empty($pickupCity) || empty($pickupPincode)) {
            $error = 'Please provide complete pickup address details.';
        } elseif (empty($preferredDate) || empty($preferredSlot)) {
            $error = 'Please select your preferred collection date and time slot.';
        } elseif (strtotime($preferredDate) < strtotime(date('Y-m-d'))) {
            $error = 'Preferred collection date must be scheduled for a future date.';
        } elseif (!$pdo) {
            $error = 'Database is offline. Unable to complete booking.';
        } else {
            try {
                // Compute theoretical recovery estimate
                $est = calculateEstimatedRecovery($pdo, $categoryId, $quantity);
                if (!$est) {
                    $error = 'Unable to compute metallurgical yield. Category may be inactive.';
                } else {
                    $requestCode = generateRequestCode();
```

Explanation and learning notes

The pickup request handler validates the CSRF token, reads the selected category and quantity, checks the address and preferred collection slot, and rejects dates in the past. It then calls the existing theoretical recovery function. These checks reduce incomplete or invalid bookings before a database insert is attempted.

D.20 Saving a Pickup Request with a Prepared Statement

Source file: user/pickup-request.php

```
            $requestCode = generateRequestCode();

            $insertStmt = $pdo->prepare("
                INSERT INTO pickup_requests (
                    request_code, user_id, center_id, device_category_id, quantity,
                    pickup_address, pickup_city, pickup_pincode, preferred_date, preferred_slot,
                    item_condition, special_notes,
                    est_copper_g, est_aluminium_g, est_silver_g, est_gold_g, est_total_value, est_co2_saved_kg,
                    is_verified, status
                ) VALUES (
                    ?, ?, ?, ?, ?,
                    ?, ?, ?, ?, ?,
                    ?, ?,
                    ?, ?, ?, ?, ?, ?,
                    0, 'Pending'
                )
            ");

            $insertStmt->execute([
                $requestCode,
                $userId,
                $centerId,
                $categoryId,
                $quantity,
                $pickupAddress,
                $pickupCity,
                $pickupPincode,
                $preferredDate,
                $preferredSlot,
                $itemCondition,
                $specialNotes,
                $est['copper_g'],
                $est['aluminium_g'],
                $est['silver_g'],
                $est['gold_g'],
                $est['total_value'],
                $est['co2_saved_kg']
            ]);

            setFlash('success', 'Pickup request successfully booked! Your booking code is <strong>' . $requestCode . '</strong>.');
            header('Location: ' . BASE_URL . '/user/view-request.php?code=' . urlencode($requestCode));
            exit;
        }
    } catch (PDOException $e) {
```

Explanation and learning notes

This SQL insert stores the generated request code, user and centre references, device category, quantity, address, schedule, estimated metal masses, estimated value, and estimated CO_2 savings. Parameter placeholders keep values separate from SQL syntax. The new request starts with is_verified set to zero and status Pending, matching the system's estimate-before-verification flow.

D.21 User Pickup History Query and Optional Status Filter

Source file: user/history.php

```
if ($pdo) {
    try {
        $sql = "
            SELECT r.*, c.category_name, c.unit_type, cc.center_name, cc.city as center_city, cc.phone as center_phone
            FROM pickup_requests r
            JOIN device_categories c ON r.device_category_id = c.id
            LEFT JOIN collection_centers cc ON r.center_id = cc.id
            WHERE r.user_id = ?
        ";
        $params = [$userId];

        if ($statusFilter !== 'all' && !empty($statusFilter)) {
            $sql .= " AND r.status = ?";
            $params[] = $statusFilter;
        }

        $sql .= " ORDER BY r.created_at DESC";

        $stmt = $pdo->prepare($sql);
        $stmt->execute($params);
        $requests = $stmt->fetchAll();
    } catch (PDOException $e) {
        $requests = [];
    }
}
```

Explanation and learning notes

The history query joins pickup requests to device categories and optionally to collection centres. It restricts results to the signed-in user, applies a status filter only when selected, and sorts newest requests first. The user ID comes from the authenticated session, which is essential to prevent one user from browsing another user’s history.

D.22 Administrator Pickup Status Update

Source file: admin/pickups.php

```
$statusFilter = $_GET['status'] ?? 'all';

// Handle Quick Status / Center Assignment Update
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!validateCsrfToken()) {
        $error = 'Security check failed. Please refresh and try again.';
    } else {
        $requestId = (int)($_POST['request_id'] ?? 0);
        $newStatus = $_POST['status'] ?? '';
        $centerId = !empty($_POST['center_id']) ? (int)$_POST['center_id'] : null;
        $adminRemarks = trim($_POST['admin_remarks'] ?? '');

        if ($requestId > 0 && !empty($newStatus)) {
            try {
                $upStmt = $pdo->prepare("
                    UPDATE pickup_requests
                    SET status = ?, center_id = ?, admin_remarks = ?
                    WHERE id = ?
                ");
                $upStmt->execute([$newStatus, $centerId, $adminRemarks, $requestId]);

                setFlash('success', 'Pickup request #' . $requestId . ' updated to status "' . sanitize($newStatus) . '".');
                header('Location: ' . BASE_URL . '/admin/pickups.php?status=' . urlencode($statusFilter));
                exit;
            } catch (PDOException $e) {
                $error = 'Status update failed: ' . $e->getMessage();
            }
        }
```

Explanation and learning notes

This administrative handler validates the form token, reads the requested status and administrative remarks, and updates the chosen pickup record through a prepared statement. Status management is separate from physical verification: changing a workflow status does not by itself prove the recovered metal weights.

D.23 Verification Results and Environmental Impact Log

Source file: admin/verify-pickup.php

```
}

// Handle Form Submission
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!validateCsrfToken()) {
        $error = 'Security check failed. Please refresh and try again.';
    } else {
        $actualWeight = (float)($_POST['actual_weight_kg'] ?? 0);
        $actualGold = (float)($_POST['actual_gold_g'] ?? 0);
        $actualSilver = (float)($_POST['actual_silver_g'] ?? 0);
        $actualCopper = (float)($_POST['actual_copper_g'] ?? 0);
        $actualAluminium = (float)($_POST['actual_aluminium_g'] ?? 0);
        $actualPayout = (float)($_POST['actual_payout_value'] ?? 0);
        $verificationNotes = trim($_POST['verification_notes'] ?? '');

        if ($actualPayout < 0) {
            $error = 'Scrap payout value cannot be negative.';
        } else {
            try {
                // Update pickup request with verified numbers
                $upStmt = $pdo->prepare("
                    UPDATE pickup_requests
                    SET is_verified = 1,
                        status = 'Completed',
                        actual_weight_kg = ?,
                        actual_gold_g = ?,
                        actual_silver_g = ?,
                        actual_copper_g = ?,
                        actual_aluminium_g = ?,
                        actual_payout_value = ?,
                        verification_notes = ?
                    WHERE id = ?
                ");
                $upStmt->execute([
                    $actualWeight,
                    $actualGold,
                    $actualSilver,
                    $actualCopper,
                    $actualAluminium,
                    $actualPayout,
                    $verificationNotes,
                    $requestId
                ]);

                // Record or update Impact Log entry
                $co2Kg = (float)$req['est_co2_saved_kg'];
                $toxicKg = (float)$req['toxic_diverted_kg'] * (float)$req['quantity'];
                $preciousG = $actualGold + $actualSilver;

                $logStmt = $pdo->prepare("
                    INSERT INTO impact_logs (pickup_id, co2_saved_kg, toxic_waste_diverted_kg, precious_metals_reclaimed_g)
                    VALUES (?, ?, ?, ?)
                    ON DUPLICATE KEY UPDATE
                        co2_saved_kg = VALUES(co2_saved_kg),
                        toxic_waste_diverted_kg = VALUES(toxic_waste_diverted_kg),
                        precious_metals_reclaimed_g = VALUES(precious_metals_reclaimed_g)
                ");
                $logStmt->execute([$requestId, $co2Kg, $toxicKg, $preciousG]);

                setFlash('success', 'Physical laboratory verification recorded successfully! Request #' . $req['request_code'] . ' is now marked as Completed.');
                header('Location: ' . BASE_URL . '/user/view-request.php?code=' . urlencode($req['request_code']));
                exit;
            } catch (PDOException $e) {
                $error = 'Failed to save verification record: ' . $e->getMessage();
```

Explanation and learning notes

The verification module records the results entered after collection or processing and writes an impact-log record. This separates estimated values from actual verified values. The application can then use verified measurements when available, while unverified requests continue to use their estimates for display. Laboratory measurements and certificates must come from authorized real-world processes, not from this code alone.

D.24 Updating Metal Benchmark Rates

Source file: admin/metal-rates.php

```
$error = '';

// Handle Rate Updates
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!validateCsrfToken()) {
        $error = 'Security check failed. Please refresh and try again.';
    } else {
        $rates = $_POST['rates'] ?? [];
        if (!empty($rates) && is_array($rates)) {
            try {
                $upStmt = $pdo->prepare("
                    UPDATE metal_rates
                    SET rate_per_gram = ?, updated_by = ?
                    WHERE id = ?
                ");

                foreach ($rates as $id => $val) {
                    $rateVal = (float)$val;
                    if ($rateVal > 0) {
                        $upStmt->execute([$rateVal, $adminId, (int)$id]);
                    }
                }

                setFlash('success', 'Benchmark metal market rates updated successfully.');
                header('Location: ' . BASE_URL . '/admin/metal-rates.php');
                exit;
            } catch (PDOException $e) {
                $error = 'Failed to update rates: ' . $e->getMessage();
            }
        }
```

Explanation and learning notes

The administrator rate module accepts rate values and updates existing records using a prepared statement. The handler checks that a submitted rate is greater than zero. A rate update changes subsequent value estimates; it does not change the amount of metal physically recovered from a device.

D.25 Database Tables for Users and Device Categories

Source file: config/seed_data.sql

```
CREATE TABLE `users` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `fullname` VARCHAR(100) NOT NULL,
  `email` VARCHAR(120) NOT NULL UNIQUE,
  `password` VARCHAR(255) NOT NULL,
  `phone` VARCHAR(20) NOT NULL,
  `address` TEXT NOT NULL,
  `city` VARCHAR(60) NOT NULL,
  `pincode` VARCHAR(10) NOT NULL,
  `role` ENUM('user', 'admin') DEFAULT 'user',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----------------------------------------------------------
-- 2. Table structure for table `collection_centers`
-- ----------------------------------------------------------
CREATE TABLE `collection_centers` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `center_name` VARCHAR(150) NOT NULL,
  `contact_person` VARCHAR(100) NOT NULL,
  `phone` VARCHAR(25) NOT NULL,
  `email` VARCHAR(100) NOT NULL,
  `address` TEXT NOT NULL,
  `city` VARCHAR(60) NOT NULL,
  `state` VARCHAR(60) NOT NULL,
  `pincode` VARCHAR(10) NOT NULL,
  `operating_hours` VARCHAR(100) DEFAULT '9:00 AM - 6:00 PM (Mon-Sat)',
  `accepted_items` TEXT COMMENT 'Comma separated device categories',
  `latitude` DECIMAL(10, 7) NULL,
  `longitude` DECIMAL(10, 7) NULL,
  `is_active` TINYINT(1) DEFAULT 1,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----------------------------------------------------------
-- 3. Table structure for table `device_categories`
-- ----------------------------------------------------------
CREATE TABLE `device_categories` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `category_name` VARCHAR(100) NOT NULL,
  `unit_type` ENUM('unit', 'kg') DEFAULT 'unit',
  `description` VARCHAR(255),
  `copper_yield_g` DECIMAL(10, 4) DEFAULT 0.0000 COMMENT 'Grams of Copper per unit or kg',
  `aluminium_yield_g` DECIMAL(10, 4) DEFAULT 0.0000 COMMENT 'Grams of Aluminium per unit or kg',
  `silver_yield_g` DECIMAL(10, 4) DEFAULT 0.0000 COMMENT 'Grams of Silver per unit or kg',
  `gold_yield_g` DECIMAL(10, 4) DEFAULT 0.0000 COMMENT 'Grams of Gold per unit or kg',
  `co2_saving_kg` DECIMAL(10, 2) DEFAULT 0.00 COMMENT 'Estimated CO2 avoided per unit or kg',
  `toxic_diverted_kg` DECIMAL(10, 2) DEFAULT 0.00 COMMENT 'Toxic material diverted from landfill',
  `is_active` TINYINT(1) DEFAULT 1
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----------------------------------------------------------
-- 4. Table structure for table `metal_rates`
-- ----------------------------------------------------------
```

Explanation and learning notes

The SQL schema defines the user records and the device categories used by the estimator. Device categories hold the per-unit yield assumptions for copper, aluminium, silver, and gold, plus environmental factors. These values are the inputs to the estimate and should be documented as benchmark assumptions, not universal yields for every physical device.

D.26 Front-End Live Estimator and Centre Search

Source file: assets/js/estimator.js

```
/**
 * Interactive Metal Recovery & Value Estimator
 * Project: E-Waste to Best
 */

document.addEventListener('DOMContentLoaded', () => {
  const categorySelect = document.getElementById('deviceCategory');
  const quantityInput = document.getElementById('deviceQuantity');
  const unitLabel = document.getElementById('unitLabel');

  // Value Display Elements
  const copperYieldDisplay = document.getElementById('copperYield');
  const aluminiumYieldDisplay = document.getElementById('aluminiumYield');
  const silverYieldDisplay = document.getElementById('silverYield');
  const goldYieldDisplay = document.getElementById('goldYield');

  const copperValueDisplay = document.getElementById('copperValue');
  const aluminiumValueDisplay = document.getElementById('aluminiumValue');
  const silverValueDisplay = document.getElementById('silverValue');
  const goldValueDisplay = document.getElementById('goldValue');

  const totalValueDisplay = document.getElementById('totalEstimatedValue');
  const co2SavingDisplay = document.getElementById('co2Saving');
  const toxicDivertedDisplay = document.getElementById('toxicDiverted');

  if (!categorySelect || !quantityInput) return;

  function formatVal(val, decimals = 3) {
    return Number(val).toLocaleString('en-IN', {
      minimumFractionDigits: decimals,
      maximumFractionDigits: decimals
    });
  }

  function calculateLive() {
    const selectedOption = categorySelect.options[categorySelect.selectedIndex];
    if (!selectedOption || !selectedOption.value) return;

    const qty = parseFloat(quantityInput.value) || 0;
    if (qty <= 0) return;

    // Extract dataset values from option element
    const copperYieldPerUnit = parseFloat(selectedOption.getAttribute('data-copper') || 0);
    const aluminiumYieldPerUnit = parseFloat(selectedOption.getAttribute('data-aluminium') || 0);
    const silverYieldPerUnit = parseFloat(selectedOption.getAttribute('data-silver') || 0);
    const goldYieldPerUnit = parseFloat(selectedOption.getAttribute('data-gold') || 0);
    const co2PerUnit = parseFloat(selectedOption.getAttribute('data-co2') || 0);
    const toxicPerUnit = parseFloat(selectedOption.getAttribute('data-toxic') || 0);
    const unitType = selectedOption.getAttribute('data-unit') || 'unit(s)';

    if (unitLabel) unitLabel.textContent = unitType;

    // Fetch market rates from window global or fallback
    const rates = window.METAL_RATES || {
      Gold: 7150.00,
      Silver: 88.50,
      Copper: 0.82,
      Aluminium: 0.24
    };

    // Compute grams
    const totalCopperG = copperYieldPerUnit * qty;
    const totalAluminiumG = aluminiumYieldPerUnit * qty;
    const totalSilverG = silverYieldPerUnit * qty;
    const totalGoldG = goldYieldPerUnit * qty;

    // Compute monetary worth
    const valCopper = totalCopperG * rates.Copper;
    const valAluminium = totalAluminiumG * rates.Aluminium;
    const valSilver = totalSilverG * rates.Silver;
    const valGold = totalGoldG * rates.Gold;
    const totalVal = valCopper + v
```

```
// Environmental offsets
const totalCo2 = co2PerUnit * qty;
const totalToxic = toxicPerUnit * qty;

// Update DOM
if (copperYieldDisplay) copperYieldDisplay.textContent = formatVal(totalCopperG, 2) + ' g';
if (aluminiumYieldDisplay) aluminiumYieldDisplay.textContent = formatVal(totalAluminiumG, 2) + ' g';
if (silverYieldDisplay) silverYieldDisplay.textContent = formatVal(totalSilverG, 3) + ' g';
if (goldYieldDisplay) goldYieldDisplay.textContent = formatVal(totalGoldG, 4) + ' g';
```

Explanation and learning notes

The JavaScript estimator reads data attributes from the selected device option, multiplies per-unit yields by quantity, applies the displayed rates, and updates page elements without a full reload. This is a user-interface calculation; the PHP backend also calculates estimates when a pickup request is submitted. The project's centre-search logic is shown in assets/js/main.js and filters cards by city and text.

APPENDIX E: FORMULAS, TERMS AND INTERPRETATION

Quick reference for explaining the project during demonstration or viva

E.1 Basic estimation formulas

Estimated metal mass (grams) = metal yield per device × quantity. Estimated value of a metal = estimated metal mass (grams) × benchmark rate per gram. Total estimated scrap value = sum of the estimated values for gold, silver, copper and aluminium. Estimated CO_2 offset = per-device CO_2 coefficient × quantity. Estimated toxic material diverted = per-device diversion coefficient × quantity. These equations explain the estimator model in the source code; they do not independently validate the scientific accuracy of the coefficients.

E.2 Important terms

- **E-waste:** Discarded electrical or electronic equipment and its parts.
- **Circular economy:** An approach that aims to keep products and materials in use through repair, reuse, refurbishment and recycling.
- **Theoretical recovery:** A calculated estimate based on assumed or reference coefficients, rather than a physical measurement.
- **Verified recovery:** A measured value recorded after collection or processing, supported by an authorized workflow or evidence.
- **Prepared statement:** A database query in which values are passed separately from the SQL command, helping prevent SQL injection.
- **CSRF token:** A session-linked random token used to help confirm that a state-changing request came from the intended form.
- **Role-based access:** A way to limit pages and actions according to whether a person is a visitor, registered citizen or administrator.

E.3 Demonstration checklist

13. Show the home page and explain the purpose of the platform.
14. Choose a device category and quantity; explain that the result is an estimate.
15. Open Awareness and Reuse Tips to demonstrate the reuse-first approach.
16. Register or sign in with a test citizen account and submit a pickup request.
17. Use a test administrator account to review the request workflow, if configured.
18. Explain how a measured recovery result should be entered separately from the estimate.
19. Show the impact dashboard and state the source and status of the displayed data.
20. Describe security helpers and the need to test all protected routes.